Деревья поведений (Behavior Trees)

Деревья поведений (Behavior Trees)

Конечные автоматы отлично работают для простых врагов: патруль -> погоня -> атака. Три состояния, пара переходов - всё ясно. Но что происходит, когда враг усложняется?

Проблема: FSM разрастается

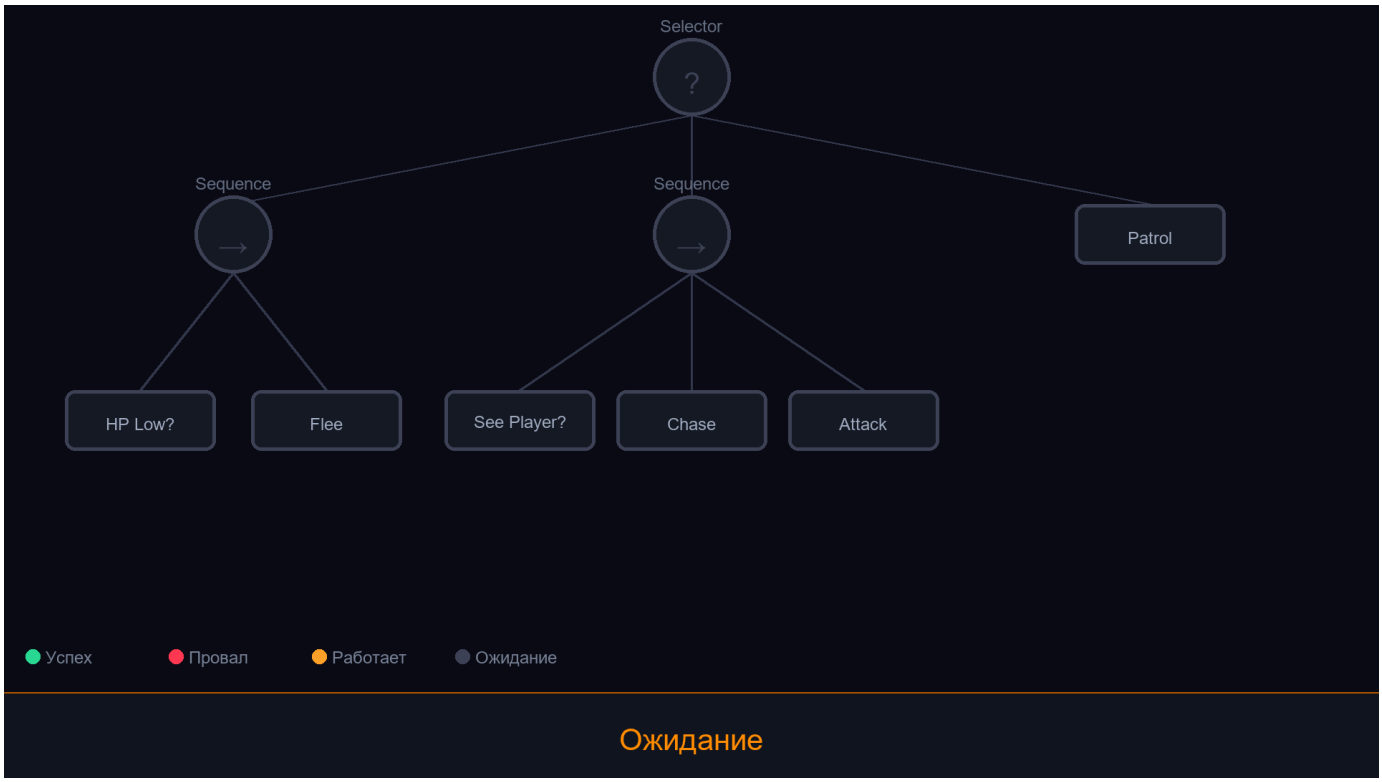
Допустим, наш охранник должен уметь:

- **Патрулировать** маршрут
- **Замечать** игрока
- **Преследовать** его
- **Атаковать**
- **Слышать шум** и идти проверять
- **Звать подкрепление**, если враг сильный
- **Убегать**, если здоровья мало
- **Возвращаться** на маршрут, если опасность миновала

Это 8 состояний. В FSM каждое состояние должно знать, в какие другие оно может перейти. Из Patrol можно перейти в Chase, Investigate, Flee. Из Chase - в Attack, Flee, CallBackup, ReturnToPost. Из Investigate - в Chase, Patrol, Flee...

Получаем паутину переходов. Добавление одного нового состояния требует пересмотра **всех существующих** - вдруг из какого-то из них тоже нужен переход в новое? Это и есть **комбинаторный взрыв переходов** (transition explosion).

А что, если вместо графа переходов - **дерево приоритетов**?



Идея: дерево решений

Behavior Tree (BT) - это дерево, где каждый узел **принимает решение** или **выполняет действие**. Каждый кадр мы проходим дерево сверху вниз, и оно определяет, что делать.

Каждый узел возвращает один из трёх статусов:

- **Success** - выполнено успешно
- **Failure** - не удалось (условие не выполнено, действие невозможно)
- **Running** - ещё выполняется (например, враг бежит к цели)

Python C#

```python
from enum import Enum


class Status(Enum):
    SUCCESS = 1
    FAILURE = 2
    RUNNING = 3


class Node:
    def execute(self, entity, world):
        raise NotImplementedError
```

Ок. Вся логика строится на комбинации узлов. Два главных типа - **Sequence** и **Selector**.

Sequence и Selector

Sequence (последовательность) - выполняет детей по порядку. Если кто-то вернул Failure - останавливается. Т.е. это **И**: "сделай A **и** B **и** C".

Selector (выбор) - пробует детей по порядку. Если кто-то вернул Success - останавливается. Т.е. это **ИЛИ**: "попробуй A, **или** B, **или** C".

Python C#

```python
class Sequence(Node):
    def __init__(self, children):
        self.children = children

    def execute(self, entity, world):
        for child in self.children:
            result = child.execute(entity, world)
            if result != Status.SUCCESS:
                return result  # Failure or Running - stop
        return Status.SUCCESS  # all children succeeded
```

Selector - зеркальный: точно такой же цикл по детям, но прерывается на **Success** вместо Failure. Если никто не успешен - возвращает Failure. Реализуйте по аналогии с Sequence, заменив условие остановки.

Всего два класса, а из них строится любое поведение.

Листья: условия и действия

Ок, внутренние узлы (Sequence, Selector) управляют **потоком** решений. Листья выполняют **реальную работу**:

- **Condition** (условие) - проверяет что-то. Возвращает Success или Failure. Никогда не возвращает статус Running.
- **Action** (действие) - делает что-то. Может вернуть Running, если действие занимает время.

Примеры ниже предполагают, что у `entity` есть атрибуты `hp`, `max_hp`, `attack_cooldown`, `damage`, `waypoints`, `current_wp` - по аналогии с ИИ врагов.

Python C#

```python
class IsPlayerVisible(Node):
    """Condition: check something, return SUCCESS or FAILURE."""
    def execute(self, entity, world):
        if can_see(entity, world.player):
            return Status.SUCCESS
        return Status.FAILURE


class ChasePlayer(Node):
    """Action: do something, may return RUNNING if not done yet."""
    def execute(self, entity, world):
        move_toward(entity, world.player.position)
        if distance(entity, world.player) < ATTACK_RANGE:
            return Status.SUCCESS
        return Status.RUNNING
```

Каждый лист - маленький, самодостаточный. Один класс - одна задача. Никакой лист не знает о структуре дерева, в которое он вставлен.

- **IsHealthLow** - condition: `hp / max_hp < threshold` → Success, иначе Failure.
- **Attack** - action: если кулдаун прошёл - нанести урон, сбросить кулдаун, вернуть Success. Иначе Failure.
- **Patrol** - action: двигаться к текущей путевой точке, при достижении - следующая. Всегда Running.
- **Flee** - action: двигаться от игрока. Всегда Running.

Собираем дерево: охранник

Теперь из этих кубиков собираем поведение:

Python C#

```python
guard_ai = Selector([
    Sequence([IsHealthLow(), Flee()]),            # priority 1: low hp -> 
    Sequence([IsPlayerVisible(), ChasePlayer(), Attack()]),  # priority 2:
    Patrol()                                       # priority 3: patrol
])

# game loop
```

```
    while running:
        guard_ai.execute(guard, world)  # return value at root is ignored
```

Читается почти как текст: "Если здоровья мало - убегай. Иначе, если видишь игрока - беги к нему и атакуй. Иначе - патрулируй."

Selector проверяет ветки **сверху вниз** - это приоритет. Побег важнее атаки, атака важнее патруля. В FSM для этого пришлось бы из каждого состояния добавлять переход "если здоровье < 30% -> Flee".

Дерево проходится с корня **каждый кадр** - не продолжает с прерванного места. Если охранник преследует (`ChasePlayer` вернул Running) и игрок спрятался - на следующем кадре `IsPlayerVisible` вернёт Failure, Sequence прервётся. Погоня остановится автоматически.

Расширяем без боли

Хотим добавить реакцию на шум? Добавляем одну ветку:

Python C#

```
guard_ai = Selector([
    Sequence([IsHealthLow(), Flee()]),
    Sequence([IsPlayerVisible(), ChasePlayer(), Attack()]),
    Sequence([HeardNoise(), InvestigateNoise()]),   # new! HeardNoise: che
    Patrol()
])
```

Одна строчка. Существующие ветки не изменились. В FSM пришлось бы добавить состояние Investigate **и** переходы из Patrol, Chase, ReturnToPost в это состояние. Чувствуете разницу?

Декораторы

Кроме Sequence и Selector есть ещё один тип внутренних узлов - **декораторы**. Грубо говоря, у декоратора один ребёнок, и он модифицирует его результат:

- **Inverter** - переворачивает: Success -> Failure, Failure -> Success. Т.е. **НЕ**.
- **Repeater** - повторяет ребёнка бесконечно (каждый тик вызывает ребёнка и возвращает Running).
- **AlwaysSucceed** - возвращает Success, даже если ребёнок вернул Failure.

Пример: `Inverter(IsPlayerVisible())` - "игрок **не** виден". Полезно для условий вроде "если безопасно - отдыхай":

Python C#

```
Sequence([Inverter(IsPlayerVisible()), Rest()])
```

Остальные два - по той же схеме, только ещё проще:

- **AlwaysSucceed** - вызывает ребёнка, игнорирует результат, всегда возвращает Success. Полезно, когда действие может провалиться, но это не должно прервать Sequence.
- **Repeater** - вызывает ребёнка, всегда возвращает Running. Превращает любое действие в бесконечное.

Одни кубики, разные деревья

Тот же набор листьев - разные деревья - разные характеры. Знакомый паттерн: одинаковый код, разные параметры.

Охранник - осторожный, приоритет на выживание:

```
Selector:
  ├─ Sequence: [IsHealthLow -> Flee]
  ├─ Sequence: [IsPlayerVisible -> Chase -> Attack]
  ├─ Sequence: [HeardNoise -> Investigate]
  └─ Patrol
```

Берсерк - атакует без оглядки:

```
Selector:
  ├─ Sequence: [IsPlayerVisible -> Chase -> Attack]
  └─ Patrol
```

Трус - убегает при виде игрока:

```
Selector:
  ├─ Sequence: [IsPlayerVisible -> Flee]
  └─ Patrol
```

Одни и те же `IsPlayerVisible`, `Chase`, `Attack`, `Flee`, `Patrol` - три совершенно разных врага. Новый тип врага = новая комбинация узлов, без нового кода.

FSM vs Behavior Tree

	FSM	Behavior Tree
Структура	Граф состояний и переходов	Дерево узлов с приоритетами
Добавить поведение	Новое состояние + переходы из/во все связанные	Новая ветка, существующие не меняются
Приоритеты	Вручную в каждом переходе	Порядок веток в Selector
Реактивность	Проверяем переходы из текущего состояния	Каждый кадр проходим дерево с корня
Сложность реализации	Проще для 3-5 состояний	Проще для 5+ вариантов поведения
Читаемость	Диаграмма состояний	Дерево читается как приоритетный список

FSM не заменяются BT. Простого врага (3 состояния) проще сделать автоматом. Сложного (7+ вариантов поведения) - деревом. Можно даже комбинировать: отдельные действия внутри BT-листьев реализовать как мини-FSM.

Итого

- **Структура** BT - это дерево решений. В ML деревья решений (decision trees) строятся автоматически из данных. В играх - проектируются вручную. Но принцип тот же: последовательность условий, которые приводят к действию.
- **Sequence = AND, Selector = OR** - это булева логика. Дерево поведений - по сути, логическое выражение в виде дерева.
- **Деревья поиска**: минимакс тоже работает с деревом - деревом возможных ходов. BT - дерево возможных действий. Разные задачи, общая идея: рекурсивный обход дерева для принятия решения.

Behavior tree, спроектированный вручную - это "экспертная система": человек закладывает правила. Следующий шаг - **обучение** дерева на данных (reinforcement learning, нейросети, MCTS)

Что	Где подробнее
FSM для простых врагов	Конечные автоматы
Patrol, Chase, Attack - реализация действий	ИИ врагов
Минимакс и деревья решений	Минимакс
Событийная модель для реакции на шум	Событийная модель
Поиск пути для ChasePlayer	Уровни и поиск путей
Компоненты для данных врага	ECS и EC
Нейросети и обучение с подкреплением	Нейросети для игр

Powered by Wiki.js